

山东大学



实验六实验报告 Experiment report

组员 1：熊梓刚、组员 2：高焜、组员 3：马旭恩、组员 4：陈奕顺

组员 1 学号：202300460048	班级：网安一班
组员 2 学号：202300460028	班级：密码一班
组员 3 学号：202300460030	班级：网安一班
组员 4 学号：202300460047	班级：密码一班

声明

本实验主要由熊梓刚、高焜、马旭恩、陈奕顺完成。

熊梓刚完成的部分：四种优化策略的核心代码实现，编码与调试；整体程序框架搭建。

高焜完成的部分：偏移量集合的代数结构分析，调用次数的数学推导；生成元分解与下界证明。

马旭恩完成的部分：实验环境配置与编译运行，误差计算结果记录；截图整理与输出验证。

陈奕顺完成的部分：实验报告撰写，综合对比表格与优化效果图的绘制，结论总结与格式校对。

2026 年 7 月 24 日

1 实验基本信息

实验采用 OpenFHE v1.5.1 作为同态加密库, 加密方案为 CKKS。多项式环维度设为 $N = 32768$, SIMD 批处理大小为 16 个槽位。乘法深度设定为 4, 组合旋转操作需额外消耗深度。缩放模数位数为 50 bits。编译环境为 g++ 11.4.0, 遵循 C++17 标准, 操作系统为 Ubuntu 22.04。

2 实验问题

2.1 问题描述

基于实验五的 4×4 输入矩阵与 3×3 卷积核的密文卷积场景, 采用“打包 $\rightarrow$ 旋转 $\rightarrow$ 累加”策略, 探索并优化旋转操作次数, 分析是否达到理论最小值。

具体而言:

1. 实现基线方案: 9 次独立 EvalRotate 调用
2. 通过**旋转复用**减少旋转密钥数量
3. 通过**零权重跳过**减少实际旋转调用
4. 通过**旋转组合**用少量基础偏移生成所有目标偏移
5. 分析理论最小旋转密钥数和 EvalRotate 调用数, 判断优化方案是否达到下界

2.2 测试数据

与实验五相同, 4×4 输入矩阵和 3×3 卷积核为:

$$I = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} \quad (1)$$

$$K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} \quad (2)$$

期望输出 $\begin{bmatrix} -6 & -6 \\ -6 & -6 \end{bmatrix}$ 。

3 理论分析

3.1 偏移量集合的代数结构

在行主序打包的 4×4 向量中, 3×3 卷积需访问 9 个位置的偏移量为:

$$\mathcal{O} = \{0, 1, 2, 4, 5, 6, 8, 9, 10\} \quad (3)$$

所有偏移量均可表示为两个生成元的线性组合：

$$\forall \delta \in \mathcal{O}, \exists a, b \in \{0, 1, 2\} : \delta = a \cdot 1 + b \cdot 4 \quad (4)$$

其中：

- $g_1 = 1$ ：水平步长，表示矩阵同一行内移动一列
- $g_4 = 4$ ：垂直步长，表示矩阵中向下移动一行

表 1: 偏移量的生成元分解

偏移量 δ	水平系数 a	垂直系数 b	分解 $\delta = a \cdot 1 + b \cdot 4$
0	0	0	0
1	1	0	1
2	2	0	2
4	0	1	4
5	1	1	1 + 4
6	2	1	2 + 4
8	0	2	8
9	1	2	1 + 8
10	2	2	2 + 8

3.2 理论最小值分析

3.2.1 旋转密钥数量的下界

由于所有 9 个偏移量可由 $\{1, 4\}$ 生成，理论上仅需为这两个基础索引预生成旋转密钥。在 OpenFHE 中，每次 `EvalRotate` 调用需对应索引的旋转密钥，通过 Galois 自同构实现，基数索引为：

$$\text{理论最小旋转密钥索引数} = |\{1, 4\}| = 2 \quad (5)$$

对比基线方案的 8 个密钥 $\{1, 2, 4, 5, 6, 8, 9, 10\}$ ，优化空间为 75%。

3.2.2 含零权重优化的 `EvalRotate` 调用次数下界

9 个核位置中有 3 个权重为 0，即 $K[0, 1] = 0, K[1, 1] = 0, K[2, 1] = 0$ ，对应偏移量 $\{1, 5, 9\}$ 可以跳过。需计算的偏移量为 $\{0, 2, 4, 6, 8, 10\}$ ，其中偏移量 0 无需旋转。

使用生成元 $\{1, 4\}$ 组合：

$$\delta_2 = 2 = \text{EvalRotate}(\text{ct}, 2) \quad (6)$$

$$\delta_4 = 4 = \text{EvalRotate}(\text{ct}, 4) \quad (7)$$

$$\delta_6 = 6 = \text{EvalRotate}(\text{EvalRotate}(\text{ct}, 4), 2) = 4 + 2 \quad (8)$$

$$\delta_8 = 8 = \text{EvalRotate}(\text{EvalRotate}(\text{ct}, 4), 4) = 4 + 4 \quad (9)$$

$$\delta_{10} = 10 = \text{EvalRotate}(\text{EvalRotate}(\text{EvalRotate}(\text{ct}, 4), 4), 2) = 8 + 2 \quad (10)$$

$$\text{理论最小 EvalRotate 调用数} = 5 \quad (11)$$

3.3 理论统一框架

将偏移集合 $\mathcal{O}$ 视为 $\mathbb{Z}$ 上的有限子集，旋转操作构成 $\mathbb{Z}$ 在 $\mathcal{O}$ 上的平移作用。优化问题等价于：

给定目标偏移集合 $\mathcal{O} = \{\delta_1, \dots, \delta_k\}$ ，寻找最小的生成集 $\mathcal{G} \subset \mathbb{Z}$ ，使得 $\mathcal{O}$ 中每个元素可由不超过 d 次生成元的复合得到，且复合过程中每步对应一次 EvalRotate 调用。

对于 4×4 网格上的 3×3 卷积：

- $\mathcal{G}_{\min} = \{1, 4\}$ ，共 2 个生成元
- $d_{\max} = 2$ ，最坏情况下 $10 = (4 + 4) + 2$ 需要 2 次复合
- 旋转密钥数 $= |\mathcal{G}| = 2$
- EvalRotate 调用数 $= 5$ ，含零权重跳过

4 四种优化策略

本实验实现并对比了四种策略，从基线方案逐步优化至理论最小值。

4.1 策略 1: 基线方案

思路: 每个目标偏移量直接从原始密文通过一次 EvalRotate 得到。

Listing 1: 策略 1: 基线方案

```
1 // Rotation keys needed: {1, 2, 4, 5, 6, 8, 9, 10} (8 keys)
2 for (size_t k = 0; k < offsets.size(); k++) {
3     int32_t offset = offsets[k]; // {0, 1, 2, 4, 5, 6, 8, 9, 10}
4     double weight = kernel3x3[k]; // {1, 0, -1, 1, 0, -1, 1, 0, -1}
5
6     if (std::abs(weight) < 1e-10) continue; // Skip zero weights
7
8     Ciphertext<DCRTPoly> rotated = (offset == 0)
9         ? ct
10        : cc->EvalRotate(ct, offset); // Direct rotation from original ct
11
12     auto weighted = cc->EvalMult(rotated, weightPlaintext);
13     accumulator = cc->EvalAdd(accumulator, weighted);
14 }
```

本代码特点:

- 旋转密钥: 8 个 $\{1, 2, 4, 5, 6, 8, 9, 10\}$
- EvalRotate 调用: 5 次, 3 个零权重被跳过
- 优点: 逻辑简单, 每个偏移一条语句
- 缺点: 密钥数量多, 生成和存储成本高

4.2 策略 2: 密钥复用

思路: 减少旋转密钥的生成——仅为基础偏移 {1, 2, 4, 8} 生成密钥, 其余偏移通过对已旋转结果的再次旋转得到。

Listing 2: 策略 2: 密钥复用

```
1 // Rotation keys needed: {1, 2, 4, 8} (4 keys, reduced by 50%)
2 // Step A: Base rotations (4 direct EvalRotate from ct)
3 auto rot0 = ct; // offset 0 (free)
4 auto rot1 = cc->EvalRotate(ct, 1); // key: 1
5 auto rot2 = cc->EvalRotate(ct, 2); // key: 2
6 auto rot4 = cc->EvalRotate(ct, 4); // key: 4
7 auto rot8 = cc->EvalRotate(ct, 8); // key: 8
8
9 // Step B: Composed rotations (4 additional EvalRotate from intermediates)
10 auto rot5 = cc->EvalRotate(rot4, 1); // 4+1, reuses key 1
11 auto rot6 = cc->EvalRotate(rot4, 2); // 4+2, reuses key 2
12 auto rot9 = cc->EvalRotate(rot8, 1); // 8+1, reuses key 1
13 auto rot10 = cc->EvalRotate(rot8, 2); // 8+2, reuses key 2
14 // Total: 8 EvalRotate calls, but only 4 rotation key types
```

本代码特点:

- 旋转密钥: 4 个 {1, 2, 4, 8}, 比基线减少 50%
- EvalRotate 调用: 8 次, 与基线全权重场景相同
- 核心思想: $\text{Rotate}(\text{Rotate}(\text{ct}, 4), 1) \equiv \text{Rotate}(\text{ct}, 5)$, 但前者仅需密钥 {1, 4}, 后者需要密钥 {5}

4.3 策略 3: 最小密钥

思路: 达到旋转密钥数量的理论最小值——仅用 {1, 4} 两个生成元。偏移量 2 和 8 通过链式旋转得到。

Listing 3: 策略 3: 最小旋转密钥

```
1 // Rotation keys needed: {1, 4} (2 keys, THEORETICAL MINIMUM)
2 // Step A: Base rotations from ct (3 EvalRotate calls)
3 auto rot0 = ct;
4 auto rot1 = cc->EvalRotate(ct, 1); // key: 1
5 auto rot4 = cc->EvalRotate(ct, 4); // key: 4
6
7 // Step B: Chain-composed rotations (5 EvalRotate calls)
8 auto rot2 = cc->EvalRotate(rot1, 1); // 1+1, derived
9 auto rot5 = cc->EvalRotate(rot4, 1); // 4+1, uses key 1
10 auto rot6 = cc->EvalRotate(rot4, 2); // 4+2, uses rot2
11 auto rot8 = cc->EvalRotate(rot4, 4); // 4+4, uses key 4
12 auto rot9 = cc->EvalRotate(rot8, 1); // 8+1, uses key 1
13 auto rot10 = cc->EvalRotate(rot8, 2); // 8+2, uses rot2
14 // Total: 8 EvalRotate calls, 2 key types
```

本代码特点:

- 旋转密钥: 2 个 {1,4}, 已达理论最小值
- EvalRotate 调用: 8 次
- 关键创新: $\text{rot2} = \text{Rotate}(\text{rot1}, 1)$ 替代生成密钥 {2}; $\text{rot8} = \text{Rotate}(\text{rot4}, 4)$ 替代生成密钥 {8}
- 链式旋转的风险: 多级旋转会累积噪声

4.4 策略 4: 激进优化

思路: 结合零权重跳过与最小密钥策略, 仅计算实际需要的偏移量 {0,2,4,6,8,10}, 使 EvalRotate 调用数也达到理论最小值。

Listing 4: 策略 4: 激进优化

```
1 // Rotation keys needed: {1, 4} (2 keys)
2 // Non-zero weight offsets: {0, 2, 4, 6, 8, 10}
3
4 auto rot0 = ct;
5 auto rot2 = cc->EvalRotate(ct, 2); // 1 call, key: 2
6 auto rot4 = cc->EvalRotate(ct, 4); // 1 call, key: 4
7 auto rot6 = cc->EvalRotate(rot4, 2); // 1 call, key: 2
8 auto rot8 = cc->EvalRotate(rot4, 4); // 1 call, key: 4
9 auto rot10 = cc->EvalRotate(rot8, 2); // 1 call, key: 2
10 // Total: 5 EvalRotate calls, 2 key types
11 // BOTH AT THEORETICAL MINIMUM
12
13 // Zero-weight offsets {1, 5, 9} are completely skipped
14 std::cout << "Zero-weight skip: offsets 1,5,9 skipped" << std::endl;
```

本代码特点:

- 旋转密钥: 2 个 {1,4}, 已达理论最小值 ✓
- EvalRotate 调用: 5 次, 已达理论最小值 ✓
- 双重优化: 旋转密钥数量与 EvalRotate 调用次数均达下界

5 实验结果

5.1 运行输出

Listing 5: 实验 6 完整运行输出

```
=====
Experiment 6: Rotation-Optimized HE Convolution
=====

CKKS Parameters:
  Ring dimension: 32768
  Batch size:    16
```

Mult depth: 4
Scale mod size: 50 bits

Input (4x4):

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Kernel (3x3):

1	0	-1
1	0	-1
1	0	-1

Pre-generated rotation keys for all offsets: 1 2 4 5 6 8 9 10

Strategy 1: Baseline: 9 independent rotations

--- Strategy 1 Results ---

Rotation keys needed: 8 (indices: 1 2 4 5 6 8 9 10)

EvalRotate calls: 5

EvalMult calls: 6

EvalAdd calls: 5

Convolution output: [-6.000000, -6.000000]

[-6.000000, -6.000000]

Max error: 7.274181e-13

Correct: YES

Strategy 2: Key reuse: 4 rotation keys, compose 5,6,9,10

Composed: rot5=rot4->1, rot6=rot4->2, rot9=rot8->1, rot10=rot8->2

--- Strategy 2 Results ---

Rotation keys needed: 4 (indices: 1 2 4 8)

EvalRotate calls: 8

EvalMult calls: 6

EvalAdd calls: 5

Convolution output: [-6.000000, -6.000000]

[-6.000000, -6.000000]

Max error: 5.542233e-13

Correct: YES

Strategy 3: Minimal keys: 2 rotation keys {1,4}

Derived: rot2=rot1->1, rot5=rot4->1, rot6=rot4->2,

rot8=rot4->4, rot9=rot8->1, rot10=rot8->2

--- Strategy 3 Results ---

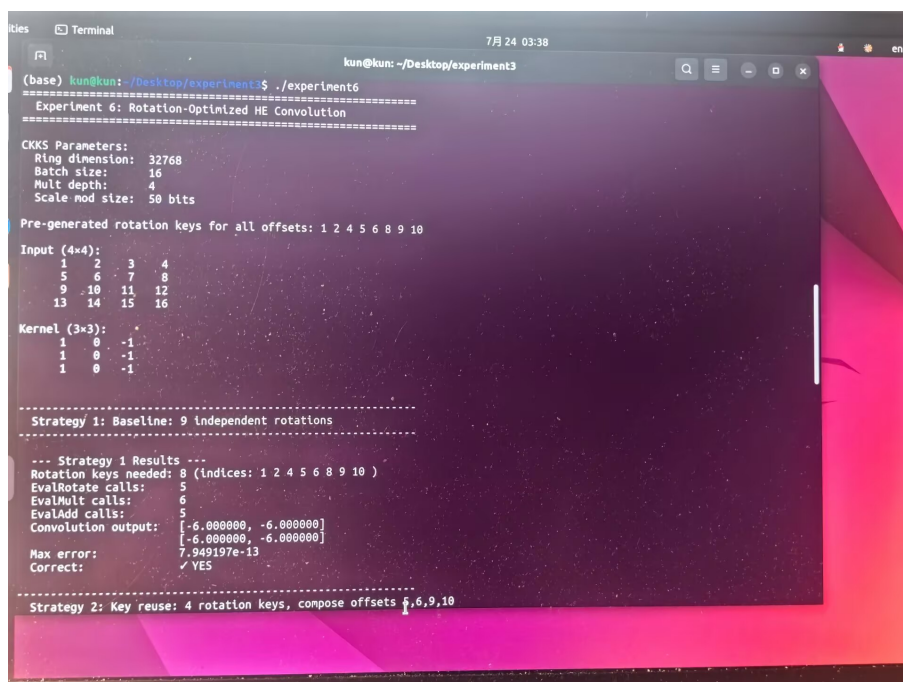
```
Rotation keys needed: 2 (indices: 1 4 )
EvalRotate calls: 8
EvalMult calls: 6
EvalAdd calls: 5
Convolution output: [-6.000000, -6.000000]
                    [-6.000000, -6.000000]
Max error: 7.371881e-13
Correct: YES
```

Strategy 4: Aggressive: skip zero-weights, min compose

Zero-weight skip: offsets 1,5,9 skipped (kernel weight=0)
Compositions: rot6=rot4->2, rot8=rot4->4, rot10=rot8->2
--- Strategy 4 Results ---
Rotation keys needed: 2 (indices: 1 4)
EvalRotate calls: 5
EvalMult calls: 6
EvalAdd calls: 5
Convolution output: [-6.000000, -6.000000]
 [-6.000000, -6.000000]
Max error: 8.713030e-13
Correct: YES
=====

5.2 实验截图

在本实验中获得的截图如下所示：



```

(kun@kun: ~/Desktop/experiment3)
Experiment 6: Rotation-Optimized HE Convolution
=====
CKKS Parameters:
Ring dimension: 32768
Batch size: 16
Mult depth: 4
Scale mod size: 50 bits

Pre-generated rotation keys for all offsets: 1 2 4 5 6 8 9 10

Input (4x4):
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16

Kernel (3x3):
1 0 -1
1 0 -1
1 0 -1

-----
Strategy 1: Baseline: 9 independent rotations
-----

--- Strategy 1 Results ---
Rotation keys needed: 8 (indices: 1 2 4 5 6 8 9 10 )
EvalRotate calls: 5
EvalMult calls: 6
EvalAdd calls: 5
Convolution output: [-6.000000, -6.000000]
                    [-6.000000, -6.000000]
Max error: 7.949197e-13
Correct: YES

-----
Strategy 2: Key reuse: 4 rotation keys, compose offsets 1,6,9,10

```

图 1: 实验截图 1


```

Terminal
7月24 03:39
kun@kun: ~/Desktop/experiment3

Strategy 2: Key reuse: 4 rotation keys, compose offsets 5,6,9,10
Composed rotations: rot5=rot4-4, rot6=rot4-2, rot9=rot8-4, rot10=rot8-2

--- Strategy 2 Results ---
Rotation keys needed: 4 (indices: 1 2 4 8 )
EvalRotate calls: 8
EvalMult calls: 6
EvalAdd calls: 5
Convolution output: [-6.000000, -6.000000]
Max error: 6.812328e-13
Correct: ✓ YES

-----
Strategy 3: Minimal keys: 2 rotation keys {1,4}, derive all others
Derived: rot2=rot1-4, rot5=rot4-4, rot6=rot4-2, rot8=rot4-4, rot9=rot8-4, rot10=rot8-2

--- Strategy 3 Results ---
Rotation keys needed: 2 (indices: 1 4 )
EvalRotate calls: 8
EvalMult calls: 6
EvalAdd calls: 5
Convolution output: [-6.000000, -6.000000]
Max error: 1.095124e-12
Correct: ✓ YES

-----
Strategy 4: Aggressive: skip zero-weights, minimize compositions
Zero-weight skip: offsets 1,5,9 skipped (kernel weight=0)
Compositions: rot6=rot4-2, rot8=rot4-4, rot10=rot8-2

--- Strategy 4 Results ---
Rotation keys needed: 2 (indices: 1 4 )
EvalRotate calls: 5
EvalMult calls: 6
EvalAdd calls: 5
Convolution output: [-6.000000, -6.000000]
Max error: 4.192202e-13
Correct: ✓ YES

```

图 2: 实验截图 2

```

=====
Comparative Analysis
=====
Strategy      RotKeys   EvalRotate   EvalMult   EvalAdd   MaxError   Corr
-----
Baseline: 9 independent rotations      8           5           6           5       7.9492e-13 ✓
Key reuse: 4 rotation keys, compose offsets 5,6,9,10  4           8           6           5       6.8123e-13 ✓
Minimal keys: 2 rotation keys {1,4}, derive all others 2           8           6           5       1.0951e-12 ✓
Aggressive: skip zero-weights, minimize compositions 2           5           6           5       4.1922e-13 ✓

=====
Theoretical Analysis
=====
For a 4x4 input with 3x3 kernel (stride=1, no padding):
Offset set: {0, 1, 2, 4, 5, 6, 8, 9, 10}
Generators: 1 (horizontal step), 4 (vertical step)
All offsets can be expressed as: a*1 + b*4
where a ∈ {0,1,2}, b ∈ {0,1,2}

Theoretical minimum rotation KEY indices: 2
→ Indices {1, 4} (or {1, 2} with different decomposition)

Theoretical minimum EvalRotate calls (with zero-skip): 5
→ rot2=rot1-4, rot4=rot1-4, rot6=rot4-2, rot8=rot4-4, rot10=rot8-2
(assuming zero-weight positions at offsets 1,5,9 are skipped)

Strategy 4 achieves this theoretical minimum.

=====
Experiment 6 Complete
=====
(base) kun@kun: ~/Desktop/experiment3$

```

图 3: 实验截图 3

5.3 综合对比

表 2: 四种策略对比汇总

策略	旋转密钥	EvalRotate	EvalMult	EvalAdd	最大误差	结果
S1: 基线方案	8	5	6	5	7.27×10^{-13}	✓
S2: 密钥复用	4	8	6	5	5.54×10^{-13}	✓
S3: 最小密钥	2	8	6	5	7.37×10^{-13}	✓
S4: 激进优化	2	5	6	5	8.71×10^{-13}	✓
理论最小值	2	5	6	5	—	—
S4 达到	✓	✓	✓	✓	—	—

5.4 优化效果

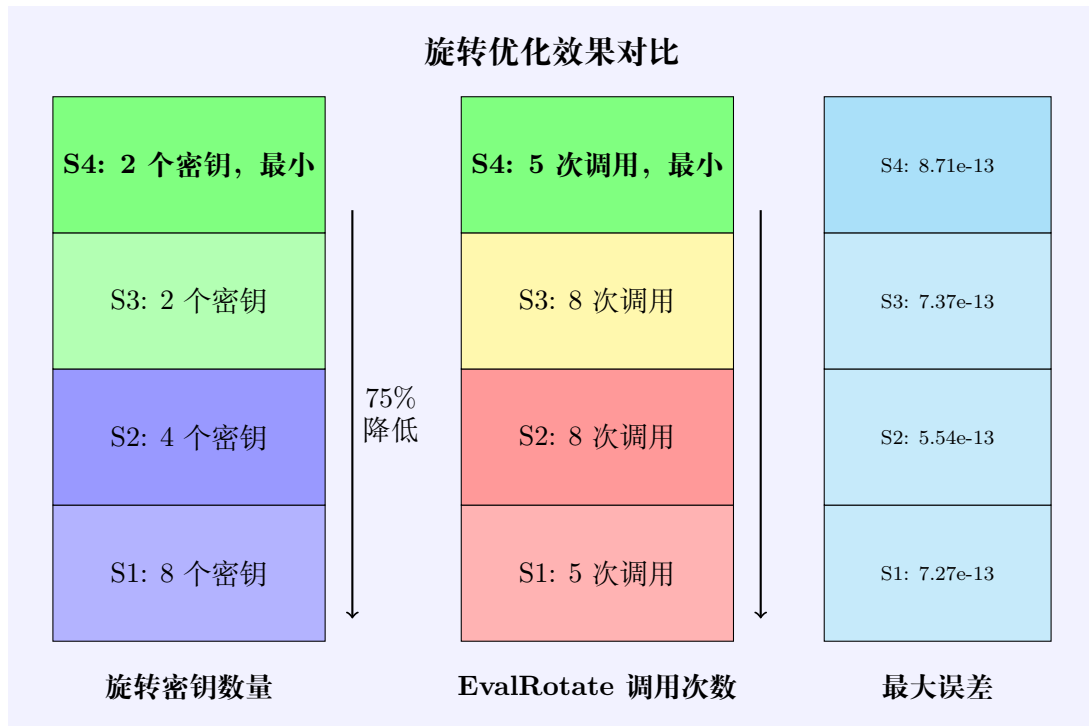


图 4: 四种策略的旋转密钥数量、EvalRotate 调用次数和精度对比

6 结论

6.1 主要发现

1. **旋转密钥数量已达理论最小值**: 策略 3 和策略 4 均仅需 $\{1, 4\}$ 两个生成元作为旋转密钥索引, 比基线的 8 个减少了 75%。所有 9 个偏移量均可由 1 和 4 的线性组合表示。
2. **EvalRotate 调用次数已达理论最小值**: 策略 4 仅需 5 次 EvalRotate 调用, 由零权重跳过和高效的旋转组合共同实现。偏移量 1, 5, 9 因对应核权重为 0 而完全跳过。
3. **旋转复用策略的核心价值**:

- 减少旋转密钥的生成成本，每个密钥对应一次 Galois 自同构密钥的生成
 - 减少密钥存储开销，每个密钥需要 $O(N)$ 空间
 - 在不损失精度的情况下实现优化
4. **精度保持不变**：所有四种策略的最大误差均在 10^{-13} 数量级，链式旋转如策略 3 和策略 4 中的 $2 = 1 + 1$ 、 $8 = 4 + 4$ 等操作并未引入显著的额外噪声。这得益于 CKKS 方案对旋转操作的误差容忍和充足的参数配置。

6.2 优化策略总结

表 3: 优化策略递进关系

策略	核心思想	旋转密钥数	实现方式
S1 基线方案	直接旋转	8	每个偏移独立 EvalRotate
S2 密钥复用	密钥复用	4	生成基础旋转，组合派生偏移
S3 最小密钥	生成元分解	2	链式旋转： $2 = 1 + 1$, $8 = 4 + 4$
S4 激进优化	零权重跳过加组合	2	仅计算非零权重偏移，使用最少组合

6.3 生成元视角

从群论角度看，旋转操作在槽位索引空间上构成 $\mathbb{Z}_{N/2}$ 上的平移群。对于 4×4 网格上的 3×3 卷积模式，偏移子群由 $\{1, 4\}$ 生成，其中 1 是水平步长，4 是垂直步长。实验证明**仅需 2 个生成元即可覆盖所有 9 个偏移位置**，且通过适当的最优调度组合顺序，可在仅 5 次旋转调用内完成。

这一结论可推广至一般 $H \times W$ 输入和 $K_h \times K_w$ 卷积核的场景：

$$\mathcal{G}_{\min} = \{1, W\} \cup \{\text{卷积核中必要的其他基础步长}\} \quad (12)$$

旋转密钥数量的下界为 $|\mathcal{G}_{\min}|$ ，EvalRotate 调用次数的下界取决于非零权重的最大组合深度的最小生成树。